

Appendix B

Setting Up and Using C on Your System

¹ Breeha Qasim, ²Bilal Wajid

Habib University

This is a short guide designed to help you get started using Unix-based systems and the C programming environment. Although it is by no means comprehensive or in-depth, it should be enough to get you started with simple C programming jobs. Before you begin, a few general considerations: you will require more than a basic knowledge of C syntax if you want to become a skilled programmer. Additionally, you should become familiar with your documentation resources, libraries, and development tools. The appropriate tools for C are `make` (a tool for organizing build processes), `gdb` (the debugger), and `gcc` (the compiler).

Although there are several libraries available, the standard library (`libc`) has a large range of capabilities and is automatically linked with all C programs; all you have to do is include the relevant header files in your code. For example, here are some commonly used standard header files in C:

```
#include <stdio.h>      // Standard Input and Output functions like printf,
scanf
#include <stdlib.h>     // Memory allocation, process control, conversions
#include <string.h>     // String handling functions
#include <math.h>       // Common mathematical functions like sqrt, pow
#include <ctype.h>      // Character classification (e.g., isdigit, isalpha)
#include <stdbool.h>    // Boolean type support
#include <time.h>       // Time and date functions
```

These headers act like roadmaps that tell the compiler where to find the function declarations and macros you'll use in your program. Most importantly, you should learn how to locate documentation. This typically entails learning how to read manual pages in Unix using the `man` command (e.g., `man printf`, `man gcc`, etc.). Finding and comprehending documentation is an essential skill that will ultimately save you time and confusion.

Learning to program successfully, especially in C, takes time and effort, just like many other useful tasks. Understanding your tools and surroundings early on will pay you greatly in the long run.



Why Learn C? Understanding C, a fundamental programming language, can help you comprehend how computers actually operate. C is used to write a large number of software libraries, embedded devices, and operating systems. Gaining knowledge about it helps you comprehend memory, performance, and the relationship between software and hardware.

I. Installing a C Development Environment

1.1 On Ubuntu (Native or via WSL)

If you are using Ubuntu directly on your machine or through WSL (Windows Subsystem for Linux), follow these steps to configure a complete C programming environment.

Step 1: Install Ubuntu or Ubuntu via WSL

If you are using Windows, install WSL and Ubuntu 22.04 from the Microsoft Store. Launch Ubuntu from the Start Menu. You'll be prompted to create a UNIX username and password for the Ubuntu environment. This user is separate from your Windows account and is used inside the Ubuntu shell.

Step 2: Update Your Package Manager

Once inside the Ubuntu terminal (WSL or native), start by updating your package list. This ensures you get the latest versions of software.

```
$ sudo apt update
```

You will be prompted to enter your password (you will not see the password as you type). Press Enter to continue.

Next, upgrade your existing software:

```
$ sudo apt upgrade
```

You may be prompted with something like:

```
$ Do you want to continue? [Y/n]
```

Type `y` and press Enter. The process might take a few minutes.

Step 3: Install Development Tools

Now install the packages necessary for C programming:

```
$ sudo apt install build-essential gdb manpages-dev
```

You may again be asked to confirm the installation. Type `y` and press `Enter` to proceed. This command installs:

- `gcc` – the GNU C compiler
- `g++` – the GNU C++ compiler (optional but included)
- `make` – a tool to help automate building programs
- `gdb` – GNU debugger
- Development headers and standard libraries

Step 4: Confirm the Installation

After installation completes, check that everything is installed correctly by typing:

```
$ gcc --version
```

You should see output similar to:

```
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE
```

To verify the other tools:

```
$ make --version
```

```
$ gdb --version
```

These commands should print the installed version numbers of the respective programs.

1.2 On macOS

Apple's macOS includes a default C compiler (`clang`) which is mostly compatible with `gcc`. However, you can install the full GNU GCC if needed for specific features or compatibility.

Step 1: Install Xcode Command Line Tools

Open Terminal and type:

```
xcode-select --install
```

This command installs:

- clang (Apple's compiler)
- make (build tool)
- Command-line tools like git
- Standard system headers required to compile C/C++ programs

You will see a prompt asking to install the command line developer tools. Confirm and allow the download to complete

Step 2: Check GCC Version

To check if a compiler was successfully installed:

```
gcc -v
```

Although you typed `gcc`, it actually points to Apple's version of `clang`. You might see:

```
gcc version Apple LLVM 14.0.3 (clang-1403.0.22.14.1)
```

This version of `clang` is compatible with most C programs. You can use it the same way as you would use `gcc`.

Step 3: Install GNU GCC (Optional)

If you need full GNU GCC (e.g., for academic or portability reasons), install Homebrew:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install GCC:

```
brew install gcc
```

This installs a version like `gcc-13`, not as `gcc`, to avoid conflict with Apple's compiler.

Step 4: Verify GNU GCC Installation

To confirm installation and locate the installed version:

```
gcc-13 --version
```

You should see:

```
gcc-13 (Homebrew GCC 13.2.0) 13.2.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

You can use this compiler explicitly when compiling:

```
gcc-13 hello.c -o hello
```

This ensures you're using the full GNU toolchain on macOS.

1.3 On Windows

Install via MinGW-w64

1. Visit the official website: <https://www.mingw-w64.org/downloads/>
2. Download the installer named mingw-w64-install.exe
3. Run the installer and configure as follows:
 - i. Architecture: x86_64 (for 64-bit systems)
 - ii. Threads: posix
 - iii. Exception: seh
 - iv. Version: Choose the latest available
 - v. Installation Directory: Set to a location you will remember, e.g., C:\mingw-w64
4. Add MinGW to your system's PATH variable so you can use its tools from any command prompt window:
 - i. Open Start Menu → Search "Environment Variables"
 - ii. Click on "Edit the system environment variables"
 - iii. In the System Properties window, click "Environment Variables"
 - iv. Under "System variables", find and select Path, then click "Edit"
 - v. Click "New" and enter the full path to MinGW's bin directory (e.g., C:\mingw-w64\bin)
 - vi. Click OK on all windows to save and apply changes
5. Open a new Command Prompt window and type:

```
gcc --version
gdb --version
```

6. Test your setup by compiling a simple C program:

```
echo "#include <stdio.h>\nint main() { printf(\"Hello, World!\\n\\n\");
return 0; }" > hello.c
gcc hello.c -o hello.exe
hello.exe
```

If everything is working correctly, you should see:

```
Hello, World!
```

2. Writing Your First C Program

Let's write one of the most basic programs we can, to get started with C. It will be stored in a file called `hello.c`. Unlike Java, for instance, where class names must correspond with file names, C does not enforce a relationship between the name of your file and its contents. Nevertheless, using descriptive filenames that accurately describe the function of your code is a smart habit.

The usage of `#include` is the first thing you'll notice in every C program. This directive is processed before the actual compilation starts because it is a preprocessor directive. Here, `#include <stdio.h>` is used to import declarations for common input and output functions, such as `printf()`, which is what we'll utilize.

The actual content of the given file is retrieved by the C preprocessor (`cpp`) and added to your code. These files reside in directories like `/usr/include/` on the majority of UNIX-like platforms.

All C programs have a `main()` function that comes after the include directive. This is the point at which execution starts. often, it accepts command-line parameters and produces an integer, often 0 for success:

- `argc` stands for "argument count" and tells you how many command-line arguments were passed.
- `argv` is an array of strings (character pointers) that holds the actual arguments. The final pointer is always `NULL`.

Here is the program:

```
/* Libraries are included here for accessing standard features */
/* This style is used for block comments */
// This is a single-line comment style

#include <stdio.h> // standard input/output functions

// main is the entry point of any C program
int main(int argc, char *argv[]) {
    // This function prints a message to standard output
    printf("hello, world");

    // Return 0 indicates the program ran without errors
    return 0;
}
```

Once compiled and run, this program will display:

```
hello, world
```

How to Create This File on Your Platform

On Ubuntu (or WSL with Ubuntu):

1. Open your terminal
2. Type `nano hello.c` to create the file.
3. Paste the above program.
4. Press `Ctrl + o` then Enter to save. Press `Ctrl + x` to exit.

On macOS:

1. Open Terminal.
2. Use `nano hello.c` or `vim hello.c`.
3. Paste the code, save and exit.

On Windows:

1. Open Notepad or VS Code.
2. Paste the code.
3. Save the file as `hello.c` in a known folder, e.g., `C:\CPrograms\hello.c`.

In the next appendix, we'll walk through compiling and running this file using a terminal or command prompt depending on your system.